



Running a Python Streamlit App in Docker on AWS EC2 – Step-by-Step (DevOps Practice)

In this blog, I containerized my **Whatsapp Chat Analyzer (Python + Streamlit)** project using Docker and deployed it on an **AWS EC2 instance**.

This was part of my hands-on DevOps practice where I worked with **Linux, Git, Docker, and AWS together in a real workflow**.

The goal was simple:

Clone repo → Create Dockerfile → Build image → Run container → Access app from browser

Repository used:

<https://github.com/teja-afk/Whatsapp-Chat-Analyzer>



Step 1 – Connect to EC2 Instance

```
ssh -i "key.pem" ubuntu@your-ec2-ip
```

Check system

```
ls
```

Create working directory

```
mkdir docker-projects
```

```
cd docker-projects
```



Step 2 – Clone GitHub Repository

```
git clone https://github.com/teja-afk/Whatsapp-Chat-Analyzer.git
```

```
ubuntu@ip-172-31-23-141:~$ mkdir docker-projects
ubuntu@ip-172-31-23-141:~$ git clone https://github.com/teja-afk/Whatsapp-Chat-Analyzer.git
Cloning into 'Whatsapp-Chat-Analyzer'...
remote: Enumerating objects: 40, done.
remote: Counting objects: 100% (40/40), done.
remote: Compressing objects: 100% (35/35), done.
remote: Total 40 (delta 16), reused 13 (delta 3), pack-reused 0 (from 0)
Receiving objects: 100% (40/40), 42.02 KiB | 6.00 MiB/s, done.
Resolving deltas: 100% (16/16), done.
ubuntu@ip-172-31-23-141:~$ cd Whatsapp-Chat-Analyzer/
```

Go inside repo

cd Whatsapp-Chat-Analyzer

ls

Files:

```
ubuntu@ip-172-31-23-141:~/Whatsapp-Chat-Analyzer$ ls
README.md  __pycache__  app.py  processes.py  requirements.txt  stop_hinglish.txt  structure.txt
```

Step 3 – Create Dockerfile

Create Dockerfile

vim Dockerfile

Add:

```
FROM python:3.11
WORKDIR /app
COPY . /app
RUN pip install --no-cache-dir -r requirements.txt
EXPOSE 8501
CMD ["streamlit", "run", "app.py", "--server.address=0.0.0.0", "--server.port=8501"]
```

Save and exit.

Step 4 – Build Docker Image

 Important: Docker image name must be lowercase

Wrong:

docker build -t Whatsapp-Chat-Analyzer .

```
ubuntu@ip-172-31-23-141:~/Whatsapp-Chat-Analyzer$ docker build -t Whatsapp-Chat-Analyzer .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

invalid argument "Whatsapp-Chat-Analyzer" for "-t, --tag" flag: invalid reference format: repository name (library/Whats
app-Chat-Analyzer) must be lowercase

Usage:  docker build [OPTIONS] PATH | URL | -
Run 'docker build --help' for more information
```

Correct:

docker build -t whatsapp-chat-analyzer .

```
ubuntu@ip-172-31-23-141:~/Whatsapp-Chat-Analyzer$ docker build -t whatsapp-chat-analyzer .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon 184.8kB
Step 1/6 : FROM python:3.11
3.11: Pulling from library/python
866771c43bf5: Pulling fs layer
ed881fbf1b07: Pulling fs layer
9da421ddeb65: Pulling fs layer
a793e3c6bce8: Pulling fs layer
8d22c1329287: Pulling fs layer
1700584b7070: Pulling fs layer
d2c5d59621cc: Pulling fs layer
```

Docker pulls python image and installs dependencies.

Successful output:

```
Step 6/6 : CMD ["python", "main.py"]
--> Running in be3535b15bc8
--> Removed intermediate container be3535b15bc8
--> 3c3d9af2b7ca
Successfully built 3c3d9af2b7ca
Successfully tagged whatsapp-chat-analyzer:latest
```

▶ Step 5 – Run Container

First I ran with wrong port mapping:

```
docker run -d -p 8000:8000 whatsapp-chat-analyzer
```

But Streamlit runs on **8501**, so container worked but app didn't open.

Stop container:

```
docker stop <container-id>
```

```
docker rm <container-id>
```

Run again with correct port:

```
docker run -d -p 8501:8501 whatsapp-chat-analyzer
```

```
ubuntu@ip-172-31-23-141:~/docker-projects/Whatsapp-Chat-Analyzer$ docker run -d -p 8501:8501 whatsapp-chat-analyzer
036d2cc360ed4ea8db1d589dad7627113c866e6fb6d55df5bdae44e77a53aca0
```

Check:

```
docker ps
```

Output:

```
ubuntu@ip-172-31-23-141:~/docker-projects/Whatsapp-Chat-Analyzer$ docker ps
```

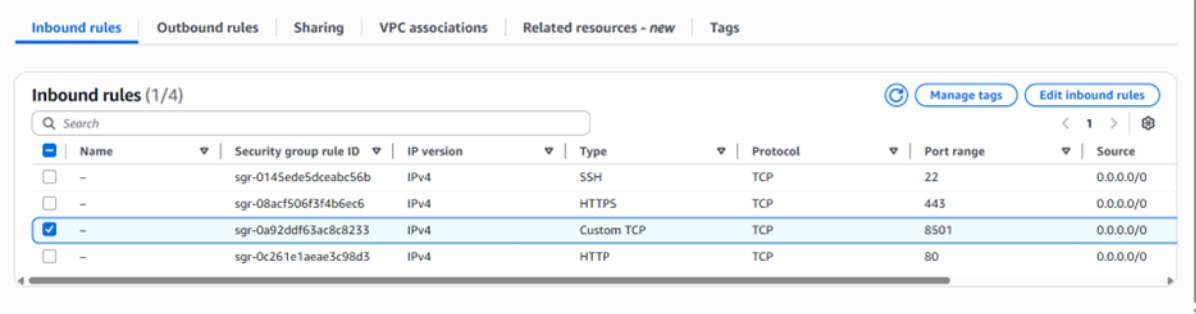
CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
036d2cc360ed	whatsapp-chat-analyzer	"streamlit run app.p..."	5 seconds ago	Up 4 seconds	0.0.0.0:8501->8501/tcp,
[...]	gifted_driscoll				

0.0.0.0:8501->8501/tcp

Step 6 – Allow Port in AWS Security Group

Go to:

EC2 → Security Groups → Inbound rules → Add rule



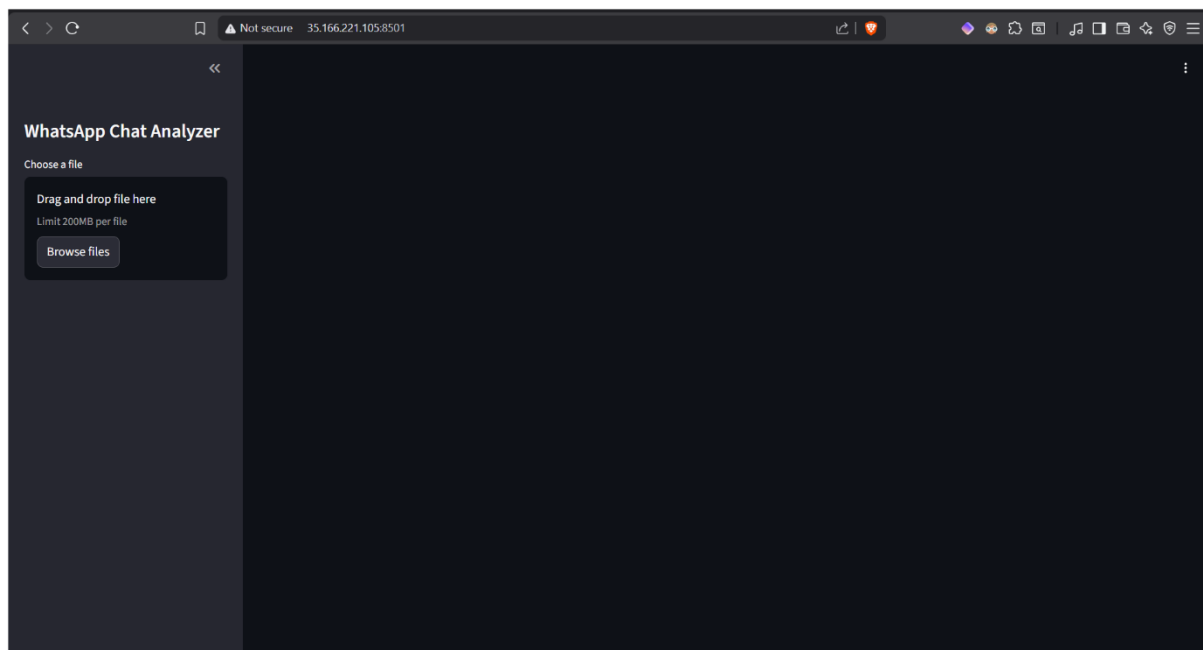
Inbound rules (1/4)								
Search								
	Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	
☐	-	sgr-0145ede5dceabc56b	IPv4	SSH	TCP	22	0.0.0.0/0	
☐	-	sgr-08acf506f3f4b6ec6	IPv4	HTTPS	TCP	443	0.0.0.0/0	
☒	-	sgr-0a92ddf63ac8c8233	IPv4	Custom TCP	TCP	8501	0.0.0.0/0	
☐	-	sgr-0c261e1aae3c98d3	IPv4	HTTP	TCP	80	0.0.0.0/0	

Save.

Step 7 – Access App in Browser

http://EC2-PUBLIC-IP:8501

Streamlit app opens successfully 🎉



Issue Faced – SSH Disconnected During Docker Run

Sometimes I got:

client_loop: send disconnect: Connection reset

This happens when:

- Docker uses CPU
- Network timeout
- SSH session closes

But container keeps running.

Check:

`docker ps`

Container was still running.

Final Workflow

- ✓ Clone repo on EC2
- ✓ Create Dockerfile
- ✓ Build Docker image
- ✓ Run container
- ✓ Fix port mapping
- ✓ Configure AWS security group
- ✓ Access app from browser

This is a real DevOps workflow used in:

- CI/CD pipelines
- Cloud deployments
- Containerized apps
- Production servers

Next Steps

Next things I will try:

- Push image to DockerHub
- Use docker-compose
- Add Nginx reverse proxy

- CI/CD with GitHub Actions
- Deploy automatically on EC2

Stay tuned.